Take-home assignment – Hotel search

Problem statement

You are required to develop a **JSON REST web service** for hotel search.
The service must have two API interfaces:

1. CRUD interface for hotel data management

Required hotel data includes:

- Hotel name
- Hotel price
- Hotel geo location

2. Search interface that returns the list of all hotels to the user

Search parameter:

- My current geo location

Output: List of hotels

- For each hotel, return the name, the price, and the distance from my current location
- The list should be ordered. Hotels that are cheaper and closer to my current location should be positioned closer to the top of the list. Hotels that are more expensive and further away should be positioned closer to the bottom of the list.

The search interface should return only the hotels prepared through the CRUD interface. You are not required to use any persistent storage (database or similar), but the design of the application should enable easy addition of the persistence layer afterwards. You'll score bonus points if the search interface supports paging.

Expected outcome

You should prepare a working proof-of-concept (PoC) solution for this assignment. At Lemax, we work with the Microsoft .NET ecosystem, so a solution written in C# and based on the .NET stack is preferred.

Demonstrating knowledge of clean architecture and domain-driven design principles is a strong plus.

As part of the assignment, we also ask you to show how you leveraged AI tools (e.g., ChatGPT, GitHub Copilot) to accelerate development and improve the quality of your solution.

Evaluation

Be sure to submit your solution before the agreed deadline. Submit it in any form you think is most appropriate. After the submission, you'll be asked to present the solution in person. We'll evaluate the solution based on the following criteria:

- **Functionality** – is the application functioning as expected? Are negative and corner cases covered?
- **Technical design** – how well does the code follow relevant design principles (OOP, Design patterns, SOLID, DRY…)? Is the code extensible and reusable?
- **Technology** – are proper tools and libraries leveraged where possible?
- **Standards** – is the API aligned with industry standards and guidelines (HTTP, REST…)?
- **Coding style** – is the coding style clean and consistent? How's the variable naming?
- **Source code organization** – are source code files organized in a folder structure according to industry best practices? Is the solution committed to a source code repository (GitHub, Bitbucket, GitLab, etc.)?
- **Performance** – what data structures and algorithms are selected? What is the complexity of the search functionality? Does it allow for scaling?
- **Security** – are secure coding practices used (defensive programming, input validation, etc.)? Does the API implement authentication and authorization?
- **Test coverage** – does the solution include unit tests? Are the test cases documented? Is test execution automated?
- **Documentation** – is the code self-documenting? Are code comments used, and for what purpose? Does the solution include markdown documentation? How easy is it for the next developer to take over this solution?
- **Processes** – does the solution include any elements of the CI/CD (build, package, test…)? How much attention was given to the application logs? Are there any other aspects implemented that would ease the usage of the application in a production environment (monitoring, health checks, etc.)?
- **Presentation skills** – how is the solution presented? Are you able to present the solution to both the technical and non-technical audience? How do you accept the feedback? How do you answer questions (good ones and bad ones)?

Don't worry, we don't expect you to score perfectly on all the points, or even to implement all of them. They are used to help us objectively assess your knowledge, experience, familiarity with relevant technologies, and your level of seniority.